Diseño de Compiladores

BNF	Reglas Semánticas
	T.t = C.t
T → BC	C.b = B.t
B → int	B.t = integer
B → float	B.t = float
C → [num] C	C.t = newarray (num, C.t)
	C.b = C.b
C → ε	C.t = C.b

Atributos

Acá no está calculando porque no respeta el orden

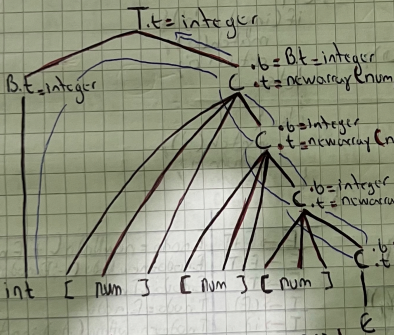
Var. sintetizadas = return
Var. heredadas = parámetro

$\Sigma = \{int, float, [,], num\}$

izquierda izquierda ⟶ Sintetizada

izquierda derecha ⟶ heredada

derecha derecha ⟶ sintetizada



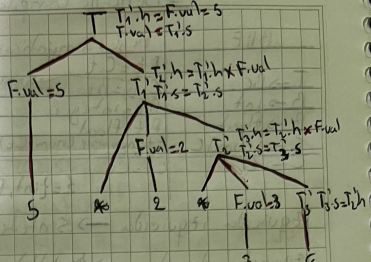

```
main(){
    T();
}

T(){
    return(C(B()));
}

C(b){
    if(input=='['){
        match('[');
        match('num');
        match(']');
        return(newarray(num,C(b)));
    }else{
        return(b);
    }
}

B(){
    if(input=='int'){
        match('int');
        return('integer');
    }else{
        match('float');
        return('float');
    }
}
```

BNF	Reglas Semánticas
	F.h = F.val
T → FT'	T'.val = T'.s
T' → * F T₁'	T₁'.h = T'.h x F.val T'.s = T₁'.s
T' → ε	T'.s = T'.h
F → 0	F.val = 0
⋮	⋮
F → 9	F.val = 9




```
T(){
    return T'(F());
}

T'(h){
    if(input == '*'){
        match('*');
        return T'(h*F());
    }else{
        return h;
    }
}
```

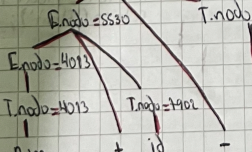
```
main(){
    printf(T());
}

F(){
    if(input == '0'){
        match('0');
        return 0;
    }else{
        match('9');
        return 9;
    }
}
```

A → X	{R.i = f(X.x)}
	{A.a = R.s}
R → Y	{R₁.i = g(Ri, Yy)}
	{R.s = R₁.s}
R → ε	{R.s = R.i}

E → E+T	E.nodo = newNodo('+', E.nodo, T.nodo)
E → E-T	E.nodo = newNodo('-', E.nodo, T.nodo)
E → T	E.nodo = T.nodo
T → (F)	T.nodo = F.nodo
T → id	T.nodo = newNodo(id, id.input)
T → num	T.nodo = newNodo(num, num.input)

Σ = {num, id, (,), -, +}

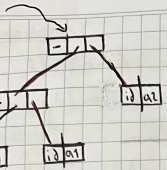


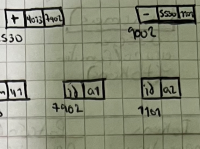
1- Rxitq : Fórmula

2- FCD: Algebra

3- FCI: Primero

E → E+T | E-T | T
E → TA A → +TA | -TA | ε





$E \rightarrow E + T \{print('+')\}$

$E \rightarrow T$	$R.i = T.nodo$
R	$E.nodo = R.s$
$R \rightarrow +T$	$R_1.i = newNodo('+', R.i, T.nodo)$
R_1	$R.s = R_1.s$
$R \rightarrow -T$	$R_1.i = newNodo('-', R.i, T.nodo)$
R_1	$R.s = R_1.s$
$R \rightarrow \epsilon$	$R.s = R.i$

i → herencia

```
E(){
  return (R(T()));
}
```

```
R(i){
  if(input=='+'){
    match('+');
    return (R(newNodo('+',i,T())));
  }else if(input=='-'){
    match('-');
    return(R(newNodo('-',i,T())));
  }else{
    return(i);
  }
}
```

$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$E.val = E.val$
$F \rightarrow digito$	$E.val = digito.x$
$digito \rightarrow 0$	$digito.x = 0$
⋮	⋮
$digito \rightarrow 9$	$digito.x = 9$

$T \rightarrow F * T \mid F$

$T \rightarrow F * T$ $\quad T.val = F.val * T.val$

$T \rightarrow F$ $\quad T.val = F.val$

$T \rightarrow FR$	$R.i = F.val$, $T.val = R.val$ } return (R(F()));
$R \rightarrow *T$	$R.val = R.i * T.val$
$R \rightarrow \epsilon$	$R.val = R.i$

$\Sigma = \{M, +, *, (,), 0, \ldots, 9\}$

Observaciones:

- El vacío 'ε' sino es heredada → inicialita.
- Cuando se pasa por referencia → no es heredada, son sintetizadas.
- Alto porcentaje de los problemas heredado se puede resolver con sintetizado.